EXPERIMENT 07

Integrating a Zoho Creator Application with Another Zoho Service Using Connections and APIs

LOW CODE APPLICATION • LABORATORY REPORT

Student Name	Aravindh Shankar
Roll Number	241401012
Programme / Year	B.Tech Computer Science and Business Systems — III Year
Institution	Rajalakshmi Engineering College
Course	Low Code Application

241401012

Experiment aim

To explain and design a secure integration between Zoho Creator and another Zoho service using connections and APIs.

Application / case context

The existing Gym Enrollment application stores member details and sends an email confirmation. A reasonable integration extension is to send approved enrollment information to Zoho CRM as a contact or lead, or to a Zoho Sheet for operational reporting. This experiment report describes the design and procedure; no CRM integration is evidenced in the supplied screenshots.

Report structure: background, design, procedure, verification, results, conclusion and references. Any feature not confirmed by supplied evidence is identified as a proposed design or procedure.

Student: Aravindh Shankar • Roll No. 241401012 • Rajalakshmi Engineering College

1. Aim, Objectives and Integration Concepts

Integration allows separate applications to exchange selected data so that users do not need to re-enter the same information. Zoho Creator can call supported Zoho services through built-in tasks, connections, APIs, or custom functions depending on the target service and account configuration.

Objectives: distinguish authentication from API calls; identify the data to share; design a field mapping; configure a connection with least privilege; handle success and failure responses; and test for duplicate or incomplete records.

Proposed scenario: after a valid gym enrollment, create or update a CRM contact using member name, email, phone, and membership plan. Sensitive health notes should not be transferred unless there is a justified requirement and appropriate access controls.

An integration is not complete merely because a connection exists. A connection provides an authorization path; the workflow or API call still needs a correct request, field mapping, response handling, and a defined outcome when the destination rejects the data. The integration should be tested from the same event that will trigger it in normal use.

Field transformations must be explicit. For example, a source field may contain a full name while the destination requires separate first and last names. Phone numbers should be sent in a consistent format, and optional fields should be omitted or handled according to the destination API's rules. Record these assumptions in a mapping table so that future changes do not silently break the integration.

Reliability requires safe retries. A network timeout can occur after the destination has already created a record, so blindly retrying may create duplicates. Use a stable record identifier or a documented duplicate check, and record the destination ID when a request succeeds. Avoid logging complete personal-data payloads when a short status and error category are sufficient.

The field map and error policy should be reviewed whenever either application changes its schema. A destination field becoming mandatory, for example, can break a previously successful integration. Document ownership of the integration and a process for resolving failed or duplicate records.

The scope of this experiment is deliberately limited to a lab-scale application. It demonstrates the underlying concept and the verification method; it does not establish production readiness without the additional checks described in this report.

24140012

Objectives and scope

This experiment studies authenticated application-to-application data exchange, correct field mapping, duplicate handling, and API failure recovery. The expected output is a documented integration design and a repeatable verification plan.

2. Architecture and Data Mapping

Source: GYM Enrollment Form in Zoho Creator. Trigger: successful record creation. Integration layer: a named connection that authorizes Creator to call the target Zoho service. Destination: Zoho CRM Contacts or Leads, depending on the business process. Response: success identifier or an error message recorded for troubleshooting.

Example mapping: Member Name to Contact Name; Member Email to Email; Member Contact to Phone; Membership Plan to a custom CRM field. Confirm required CRM fields before sending data. If using a custom API, send only fields accepted by the destination endpoint.

Authentication should use a supported connection/OAuth flow rather than hard-coded credentials in a Deluge script. Grant only the scopes needed to create or update the intended records and review who can edit the connection.

An integration is not complete merely because a connection exists. A connection provides an authorization path; the workflow or API call still needs a correct request, field mapping, response handling, and a defined outcome when the destination rejects the data. The integration should be tested from the same event that will trigger it in normal use.

Field transformations must be explicit. For example, a source field may contain a full name while the destination requires separate first and last names. Phone numbers should be sent in a consistent format, and optional fields should be omitted or handled according to the destination API's rules. Record these assumptions in a mapping table so that future changes do not silently break the integration.

Reliability requires safe retries. A network timeout can occur after the destination has already created a record, so blindly retrying may create duplicates. Use a stable record identifier or a documented duplicate check, and record the destination ID when a request succeeds. Avoid logging complete personal-data payloads when a short status and error category are sufficient.

The field map and error policy should be reviewed whenever either application changes its schema. A destination field becoming mandatory, for example, can break a previously successful integration. Document ownership of the integration and a process for resolving failed or duplicate records.

Decide what the source application should do if the destination is unavailable. For a non-critical CRM synchronization, the enrollment may still be saved while the integration failure is queued for retry. For a decision-critical service, the business may instead need to stop the operation and show a clear message. This policy must be deliberate.

Use a unique source record identifier as a correlation value wherever possible. It helps administrators trace one enrollment across Creator, a custom function, and the target service without relying on a member name that may not be unique.

An integration should also define ownership. Decide who monitors failed calls, who can reauthorize a connection, and how a failed item is retried. If the integration has no owner, errors can accumulate silently and the source and destination records may diverge.

During testing, use synthetic member details and verify the exact destination record. Confirm that fields are not shifted or overwritten and that the destination has the intended owner and visibility. Keep the API response or record ID as evidence, while redacting any secret values.

Design / configuration summary

Test case	Expected outcome
Valid record	Destination record created or updated once
Duplicate email	Duplicate policy applied; no unintended duplicate
Missing required destination field	Clear error; failure logged
Expired/revoked connection	Request fails safely; connection can be reauthorized
Sensitive field	Excluded unless approved and necessary

3. Configuration Procedure

In Creator, open Connections or the integration configuration area and create a connection for the target Zoho service. Sign in, select only the necessary scopes, and give the connection a meaningful name such as gym_crm_connection.

Configure the workflow trigger to run after a valid record is created. Use the appropriate built-in task or invoke the target API using the authenticated connection. Map the fields explicitly and check whether the destination expects a single name field or separate first and last name fields.

Define duplicate handling: use a stable identifier such as email where appropriate to search before creating a new contact. Decide whether updates are permitted, and avoid overwriting existing CRM data unintentionally.

An integration is not complete merely because a connection exists. A connection provides an authorization path; the workflow or API call still needs a correct request, field mapping, response handling, and a defined outcome when the destination rejects the data. The integration should be tested from the same event that will trigger it in normal use.

Field transformations must be explicit. For example, a source field may contain a full name while the destination requires separate first and last names. Phone numbers should be sent in a consistent format, and optional fields should be omitted or handled according to the destination API's rules. Record these assumptions in a mapping table so that future changes do not silently break the integration.

Reliability requires safe retries. A network timeout can occur after the destination has already created a record, so blindly retrying may create duplicates. Use a stable record identifier or a documented duplicate check, and record the destination ID when a request succeeds. Avoid logging complete personal-data payloads when a short status and error category are sufficient.

The field map and error policy should be reviewed whenever either application changes its schema. A destination field becoming mandatory, for example, can break a previously successful integration. Document ownership of the integration and a process for resolving failed or duplicate records.

Decide what the source application should do if the destination is unavailable. For a non-critical CRM synchronization, the enrollment may still be saved while the integration failure is queued for retry. For a decision-critical service, the business may instead need to stop the operation and show a clear message. This policy must be deliberate.

Use a unique source record identifier as a correlation value wherever possible. It helps administrators trace one enrollment across Creator, a custom function, and the target service without relying on a member name that may not be unique.

An integration should also define ownership. Decide who monitors failed calls, who can reauthorize a connection, and how a failed item is retried. If the integration has no owner, errors can accumulate silently and the source and destination records may diverge.

During testing, use synthetic member details and verify the exact destination record. Confirm that fields are not shifted or overwritten and that the destination has the intended owner and visibility. Keep the API response or record ID as evidence, while redacting any secret values.

Procedure and configuration checklist

- Create a least-privilege connection for the destination service.
- Document source-to-target field mappings.
- Implement the call and response/error handling.
- Verify both Creator and destination records.

4. API Reliability, Security and Testing

An API request can fail because of authentication expiry, missing required fields, invalid payload format, rate limits, network errors, or permissions. The workflow should handle errors and avoid telling the user an integration succeeded unless the destination confirms it.

Test a valid enrollment, missing email, duplicate email, revoked connection, and destination validation failure. Verify both sides: the Creator record and the destination record. Check logs for response codes and redact personal data from diagnostic messages.

Use least privilege, restrict access to connections, avoid sending health-related notes without need, and document data retention. An API integration introduces a second data store, so deletion and correction policies must be considered.

An integration is not complete merely because a connection exists. A connection provides an authorization path; the workflow or API call still needs a correct request, field mapping, response handling, and a defined outcome when the destination rejects the data. The integration should be tested from the same event that will trigger it in normal use.

Field transformations must be explicit. For example, a source field may contain a full name while the destination requires separate first and last names. Phone numbers should be sent in a consistent format, and optional fields should be omitted or handled according to the destination API's rules. Record these assumptions in a mapping table so that future changes do not silently break the integration.

Reliability requires safe retries. A network timeout can occur after the destination has already created a record, so blindly retrying may create duplicates. Use a stable record identifier or a documented duplicate check, and record the destination ID when a request succeeds. Avoid logging complete personal-data payloads when a short status and error category are sufficient.

The field map and error policy should be reviewed whenever either application changes its schema. A destination field becoming mandatory, for example, can break a previously successful integration. Document ownership of the integration and a process for resolving failed or duplicate records.

Decide what the source application should do if the destination is unavailable. For a non-critical CRM synchronization, the enrollment may still be saved while the integration failure is queued for retry. For a decision-critical service, the business may instead need to stop the operation and show a clear message. This policy must be deliberate.

Use a unique source record identifier as a correlation value wherever possible. It helps administrators trace one enrollment across Creator, a custom function, and the target service without relying on a member name that may not be unique.

An integration should also define ownership. Decide who monitors failed calls, who can reauthorize a connection, and how a failed item is retried. If the integration has no owner, errors can accumulate silently and the source and destination records may diverge.

During testing, use synthetic member details and verify the exact destination record. Confirm that fields are not shifted or overwritten and that the destination has the intended owner and visibility. Keep the API response or record ID as evidence, while redacting any secret values.

Verification checklist

- Verify authorization and destination field mapping.
- Test duplicate requests and failed responses.
- Confirm the destination state matches the source.

5. Results and Conclusion

The proposed design shows how Creator can exchange enrollment data with another Zoho service through a secure connection and API. It can reduce re-entry and support follow-up, but introduces synchronization, duplicate-handling, and privacy responsibilities.

No CRM connection or API call is shown in the supplied screenshots; therefore the report treats the integration as the design and implementation procedure for this experiment, not as an already completed feature.

Conclusion: integrations should be designed around a clear business purpose, minimal field mapping, controlled authentication, explicit error handling, and tests that verify the destination—not merely the source workflow.

An integration is not complete merely because a connection exists. A connection provides an authorization path; the workflow or API call still needs a correct request, field mapping, response handling, and a defined outcome when the destination rejects the data. The integration should be tested from the same event that will trigger it in normal use.

Field transformations must be explicit. For example, a source field may contain a full name while the destination requires separate first and last names. Phone numbers should be sent in a consistent format, and optional fields should be omitted or handled according to the destination API's rules. Record these assumptions in a mapping table so that future changes do not silently break the integration.

Reliability requires safe retries. A network timeout can occur after the destination has already created a record, so blindly retrying may create duplicates. Use a stable record identifier or a documented duplicate check, and record the destination ID when a request succeeds. Avoid logging complete personal-data payloads when a short status and error category are sufficient.

The field map and error policy should be reviewed whenever either application changes its schema. A destination field becoming mandatory, for example, can break a previously successful integration. Document ownership of the integration and a process for resolving failed or duplicate records.

A good conclusion separates observed results from expected results. Report any failed test and the next corrective action plainly; a documented limitation is more useful than claiming a pass that the evidence does not support.

24110042

Learning outcomes

- Explained connections, API requests and field mapping.
- Identified duplicate and failure-handling requirements.
- Designed a secure, verifiable integration path.

Conclusion

Conclusion: integrations should be designed around a clear business purpose, minimal field mapping, controlled authentication, explicit error handling, and tests that verify the destination—not merely the source workflow.

References

Zoho Creator API documentation: https://www.zoho.com/creator/help/api/

Zoho Creator connections overview: https://www.zoho.com/creator/help/connections/

Zoho CRM API documentation: https://www.zoho.com/crm/developer/docs/api/v8/

6. Integration Design: Mapping, Idempotency and Observability

Field mapping should be written down before coding. For every source field, record the target field, format, whether it is required, and whether it contains personal data. Transform data only when the transformation is deterministic and documented.

Idempotency prevents a retry from creating multiple destination records for one enrollment. Use a stable source record ID or another suitable unique key and define what should happen if a record already exists. Do not rely on a display name as a unique identifier.

Operational observability should capture the action, timestamp, result, and a safe error description. Avoid logging full payloads containing phone numbers, addresses, or medical notes. Provide a process for reauthorizing a connection and retrying failed records safely.

Decide what the source application should do if the destination is unavailable. For a non-critical CRM synchronization, the enrollment may still be saved while the integration failure is queued for retry. For a decision-critical service, the business may instead need to stop the operation and show a clear message. This policy must be deliberate.

Use a unique source record identifier as a correlation value wherever possible. It helps administrators trace one enrollment across Creator, a custom function, and the target service without relying on a member name that may not be unique.

Practical application to the lab

The existing Gym Enrollment application stores member details and sends an email confirmation. A reasonable integration extension is to send approved enrollment information to Zoho CRM as a contact or lead, or to a Zoho Sheet for operational reporting. This experiment report describes the design and procedure; no CRM integration is evidenced in the supplied screenshots.

Evaluation criteria

A satisfactory lab result should be reproducible, supported by a clear configuration or test record, and honest about limitations. Record the setup, test input, expected result, actual result, and evidence. If a feature was not enabled or tested in the current account, report that as a limitation and describe the procedure needed to complete it.

Checklist for submission

- Confirm the report matches the experiment title and stated objectives.
- Keep screenshots and configuration details aligned with what was actually performed.
- Separate verified outcomes from proposed implementation steps.
- Recheck the final PDF page layout before submission.

7. Integration Test Plan and Final Recommendations

Integration tests should verify source-to-target field mapping, required fields, duplicates, permission failures, and recovery from temporary errors. A single successful API call is not enough to establish reliable synchronization.

Record the target record identifier or another traceable reference after success. For failure, record the safe error category and provide a retry or manual review process. Reconcile the source and destination periodically if the business depends on the integration.

Final recommendation: begin with the smallest useful integration, avoid unnecessary personal data, and expand only after the first path is reliable.

Decide what the source application should do if the destination is unavailable. For a non-critical CRM synchronization, the enrollment may still be saved while the integration failure is queued for retry. For a decision-critical service, the business may instead need to stop the operation and show a clear message. This policy must be deliberate.

Use a unique source record identifier as a correlation value wherever possible. It helps administrators trace one enrollment across Creator, a custom function, and the target service without relying on a member name that may not be unique.

Practical application to the lab

The existing Gym Enrollment application stores member details and sends an email confirmation. A reasonable integration extension is to send approved enrollment information to Zoho CRM as a contact or lead, or to a Zoho Sheet for operational reporting. This experiment report describes the design and procedure; no CRM integration is evidenced in the supplied screenshots.

Evaluation criteria

A satisfactory lab result should be reproducible, supported by a clear configuration or test record, and honest about limitations. Record the setup, test input, expected result, actual result, and evidence. If a feature was not enabled or tested in the current account, report that as a limitation and describe the procedure needed to complete it.

Checklist for submission

- Confirm the report matches the experiment title and stated objectives.
- Keep screenshots and configuration details aligned with what was actually performed.
- Separate verified outcomes from proposed implementation steps.
- Recheck the final PDF page layout before submission.

24114012